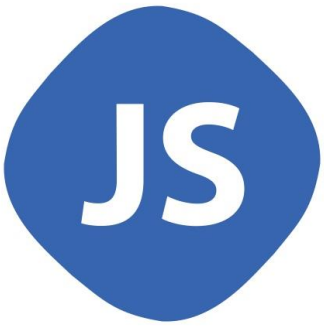


Преобразование типов

- Введение
- Явное и неявное преобразование
- Преобразование к типу String
- Преобразование к типу Number
- Преобразование к типу Boolean
- Функции parseInt и parseFloat
- Операторы вместо функций

Введение

В предыдущем уроки мы изучили **типы данных в JavaScript**.

JavaScript различает типы данных:

- **Примитивные**.
- **Ссылочнные**.

Существуют типы данных, которые могут содержать значения:

- **Строка** (String).
- **Числовой** (Number).
- **Логический** (Boolean).
- **Объект** (Object).

Есть объектные типы:

- **Объект** (Object).
- **Массив** (Array).

Есть типы данных, которые не могут содержать значения:

- **Нулевой** (Null).
- **Неопределенный** (Undefined).





Чтобы определить тип переменной, в JavaScript используется оператор **typeof**.

```
console.log (typeof 3); // number
```

JavaScript – язык с **динамической типизацией** данных. Это значит, что в одну и ту же переменную можно записывать значения **различных типов**, при этом тип самой переменной **будет меняться**. Автоматическое преобразование типов является одной из самых мощных возможностей JavaScript.

В этом уроке научимся преобразовывать примитивные типы данных JavaScript, числа, строки, логические элементы.

Но прежде чем начать, приведу простой пример для тех, у кого JavaScript первый в списке изучаемых языков программирования и кто не понимает, а зачем это вообще надо.

Тип данных определяет возможные операции над операндами.

Например:

2 + 2 = 4 – это операция сложения **чисел**.

"2" + "2" = "22" – это уже операция **конкатенации** (сложения) **строк**, которые, ксати, очень похожи на числа. Про такие строки говорят, что это строковое представление числа.

Пока все достаточно логично. Но, что будет в этом примере:

"2" + 2 = ? – JS расценит это как операцию сложения или конкатенации?

А в этом:`

("2" + 2) / 10 = ? - ??

Пробуем на примере.

example_1. Перед выполнением операции JS проверяет типы операндов

```
<script>

      // преобразование типов способно преподносить сюрпризы
```

```
    console.log ("2 + 2 не равно '2' + '2'");
    // здесь логично
    console.log ("2 + 2 = ", 2 + 2 );  // 4
    // здесь вроде бы тоже
    console.log ("'2' + '2' = ", '2' + '2'); // "22"
    // но, что будет здесь?
    console.log ("'2' + 2 = ", '2' + 2);  // "22"
    // а здесь?
    console.log ("('2' + 2) / 10 = ", ('2' + 2) / 10);  // 2.2
    // всегда задавайте себе вопросы
</script>
```

Совет. Всегда задавайте себе вопросы и пытайтесь найти на них ответы. Все интересующие вас явления в языке программирования должны получить **какое-то объяснение**. Будет ли оно всегда верным? Нет, конечно. С изучением технологии для вас всегда будут открываться новые истины. Однажды придуманная теория будет трансформироваться и преобретать все новые очертания. Но ...

Мыслительный процесс – бесценен.

Явное и неявное преобразование

Преобразование типов данных может быть **явным** и **неявным**. Так как JavaScript – это слабо типизированный или динамический язык программирования, значения могут быть конвертированы между различными типами **автоматически**. Это называют **неявным приведением типов**. Обычно неявное преобразование происходит, когда в **выражениях** используют значения различных типов.



Неявное приведение типов

Когда JavaScript приходится оперировать с "неправильным" типом данных, интерпретатор попытается преобразовать тип этого значения в "правильный".

Результат получается не всегда таким, как вы ожидаете. Если по примеру кода листинга example_1 вам показалось, что вы поняли как это работает, вот еще несколько примеров неявного образования.

example_2. Неявное преобразование типов

```
<script>
    // неявное преобразование
    // 2 преобразуется в "2"
    console.log("5" + 2);  // 52
    // но,
    // "5" преобразуется в 5
    console.log("5" - 2);  // 3
    // "5" и "2" преобразуется в 5 и 2
    console.log("5" * "2");   // 10
    // null преобразуется в 0
    console.log(5 + null);    // 5
    // но,
    // null преобразуется в "null"
    console.log("5" + null);  // 5null
</script>
```

Явное приведение типов

Явное преобразование – это когда преобразование типов данных выполняет **сам программист**. Явное преобразование иначе называют **приведением типов**.

example_3. Явное преобразование типов

```
<script>
    // явное преобразование
    // строку '2' к числу 2
    console.log (Number('2') + 2);  // 4
    // явное преобразование
    // число 2 к строке '2'
    console.log (String(2) + String(2)); // 22
</script>
```

Для явного преобразования в примитивные типы данных в JavaScript используются следующие функции: **Boolean**(), **Number**(), **String**(). При неявном преобразования интерпретатор JavaScript автоматически использует те же самые функции, что используются и для явного преобразования.

Таким образом. Переменные JavaScript могут быть преобразованы в новые переменные и другие типы данных:

- при помощи специальных **функций** JavaScript;
- **автоматически** самим JavaScript.

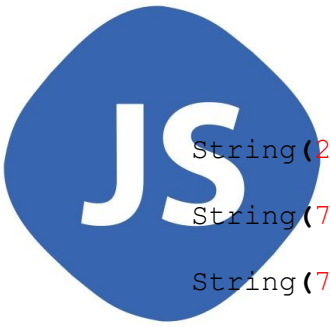
Преобразование к типу String

Для того чтобы явно преобразовать значение в строку, можно воспользоваться функцией **String**().

Преобразование чисел

Функцию **String** можно использовать с любыми видами чисел — константами, переменными или выражениями.

```
// преобразование чисел
```



```
String(21);  // "21"

String(7.6);  // "7.6"

String(7 + 23);  // "30"
```

Преобразование логических значений

String позволяет преобразовывать логические значения в строки.

```
// преобразование логических значений

String(false);  // "false"

String(true);  // "true"
```

Преобразование даты

String позволяет преобразовывать дату (тип Date) в строку.

```
// преобразование даты

String(new Date());  // "Fri Jul 15 2022 12:24:08 GMT+0300 (Москва, стандартное время)"
```

example_4. Преобразование к типу String

```
<script>

	// приведение к строке

	console.log(String(21));  // "21"

	console.log(String(-77.7));  // "-77.7"

	console.log(null);  // null

	console.log(String(null));  // "null"

	console.log(undefined);  // undefined

	console.log(undefined + "");  // "undefined"

	console.log(true + false);  // 1

	console.log(true + false + "");  // "1"

	// можно проверить
```

```
    console.log(typeof(true + false));  // number
    console.log(typeof(true + false + ""));  // string
</script>
```

Обратите внимание. Для **неявного преобразования** применяется оператор **+**, у которого **один из аргументов строка**. В этом случае **+** приводит к строке и другой аргумент.

```
console.log(21 + "");  // "21"
```

Преобразование к типу Number

В Javascript явное преобразование примитивных значений к числу (не обязательно целому, может быть и дробное) осуществляется с помощью функции **Number**().

Преобразование значений с помощью функции Number() осуществляется по следующим правилам:

- Числа возвращаются без каких либо изменений.
- Значение **null** преобразуется в 0.
- Значение **undefined** преобразуется в NaN.

```
Number("25");  // 25
Number(null);  // 0
Number(undefined);  // NaN
```

Преобразование строк

Преобразование строк осуществляется по правилам:

- При преобразовании строк в числа интерпретатор обрезает пробелы, а также управляющие символы (перенос строки **\n**, табуляция **\t**,

вертикальная табуляция **\v** и др.), которые находятся в начале или в конце строки.

- Если строка содержит только цифры с унарным оператором **-** или **+** либо без знака, она всегда преобразуется в целое десятичное число (отрицательное или положительное). Начальные нули игнорируются, например "007" преобразуется в 7.
- Если строка представляет собой число в шестнадцатеричном формате, она преобразуется в соответствующее целое десятичное число.
- Пустая строка преобразуется в 0.
- Если строка содержит что-то отличное от предыдущих вариантов, например инкремент / декремент (будем изучать позже), она преобразуется в NaN.

```
Number("  25 \n");  // 25
Number("-0025.3  ");  // -25.3
Number("0xA");  // 10
Number("    ");  // 0
Number("++2");  // NaN
```

Преобразование логических значений

Функция **Number** позволяет преобразовывать логические значения в числа

```
Number(true);  // 1
Number(false);  // 0
```

Преобразование даты

Функция **Number** позволяет преобразовывать значения даты (тип Date) в числа.

```
Number(new Date());  // 1657882426358
```

example_5. Преобразование к типу Number

```
<script>
```

```
    // приведение к числу
    console.log(Number("25"));  // 25
    console.log(Number(null));  // 0
    console.log(Number(undefined));  // 0
    console.log(Number("  25 \n"));  // 25
    console.log(Number("-0025.3  "));  // -25.3
    console.log(Number("0xA"));  // 10
    console.log(Number("    "));  // 0
    console.log(Number("++2"));  // NaN
    console.log(Number(true));  // 1
    console.log(Number(false));  // 0
    console.log(Number(new Date()));  // 1657882426358
</script>
```

Неявное приведение значения к числовому типу в JavaScript применяется, чаще чем преобразование в строку или в логическое значение. Преобразование к типу Number выполняют следующие операторы:

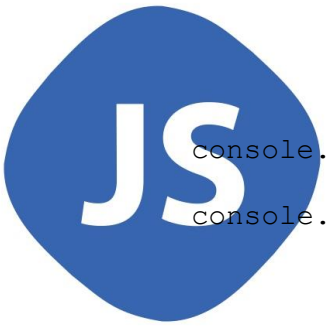
- Операторы сравнения <, >, <=, >=.
- Арифметические операторы +, -, *, /, %.
- Унарный оператор +.
- Побитовые операторы ^, |, &, ~.
- Операторы нестрогого равенства == и неравенства !=.

Обратите внимание. При этом не все так очевидно.

Оператор **+** с двумя операндами не вызывает неявное преобразование к числовому типу, если **хотя бы один** оператор является **строкой**.

Оператор **==** не производит неявного преобразования в число, если оба операнда являются строками

```
console.log ("5" + 6);  // "56"
```



```
console.log ("5" == 5.0);  // true
console.log ("5" == "5.0");  // false
```

Преобразование к типу Boolean

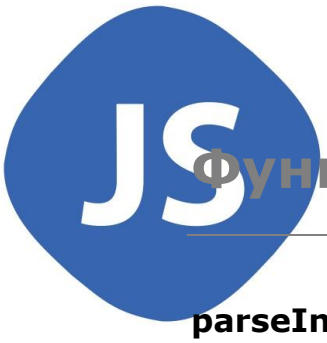
Для того, чтобы явно преобразовать значение к логическому типу, используют функцию **Boolean**().

- Следующие значения в результате преобразования дают логическое **false**: **undefined**, **null**, **0**, **-0**, **NaN**, **""**.
- Значение **false** возвращается без изменений.
- **Все остальные значения** в результате преобразования дают логическое **true** (включая объекты, функции, массивы, даты, значения типа Symbol, пустые объекты и пустые массивы).

example_6. Преобразование к типу Boolean

```
<script>
    // приведение к логическому типу
    console.log(Boolean(''));  // false
    console.log(Boolean(0));  // false
    console.log(Boolean(NaN));  // false
    console.log(Boolean(null));  // false
    console.log(Boolean(undefined));  // false
    console.log(Boolean(false));  // false
    console.log(Boolean("привет"));  // true
    console.log(Boolean("12345"));  // true
</script>
```





Функции parseInt и parseFloat

parseInt

Для преобразования строки в **целое число** в JavaScript есть еще замечательная функция **parseInt**():

```
console.log(parseInt("25") + 5); // 30
```

Все работает, все хорошо. Только пока непонятно, зачем городить огород новой функцией, если действие абсолютно равнозначно действию функции Namber.

А дело в том, что **Number** выполняет **преобразование типов**, **parseInt** выполняет **синтаксический анализ**.

> ***К сведению***. ParseInt – от слова **parser** (парсер). Парсером называют программу, которая занимается **синтаксическим анализом** входных данных по определённым правилам с целью их **сбора и систематизации**.

Функция parseInt() **анализирует** строковый аргумент и возвращает целое число в указанной системе счисления.

Аргумент может иметь смешанное содержимое, например, "123hello", то есть в данном случае есть цифры, но есть и обычные символы. В таком случае, parseInt() все равно попытается выполнить преобразование - она последовательно, начиная с первого символа, считывает цифры, пока не встретит первый нецифровой символ.

Если функции parseInt() не удастся выполнить преобразование, она возвращает значение **NaN** (Not a Number), которое говорит о том, что строка не представляет число и не может быть преобразована.

> ***Важно***. ParseInt начинает анализ с того, что преобразует аргумент в строку, анализирует эту строку, а затем возвращает **целое число** или **NaN**.



example_7. Тестируем parseInt

```
<script>
	// Number не приводит тип
	console.log(Number("123qwerty")); // NaN
	// parseInt справляется
	console.log(parseInt("123qwerty")); // 123
	console.log(parseInt("123qwerty") + 3); // 126
	// но тоже не всегда
	console.log(parseInt("qwerty123")); // NaN
</script>
```

parseFloat

Для преобразования строк в **дробные числа** применяется функция parseFloat(), которая работает аналогичным образом.

example_8. Тестируем parseFloat

```
<script>
	console.log(parseInt("123.8qwerty")); // 123
	console.log(parseFloat("123.8qwerty") + 3); // 126.8
</script>
```

Операторы вместо функций

Вместо функций для явного преобразования типов данных можно использовать **операторы**.

Например, если один из операндов оператора сложения **+** является строкой, то другой операнд также преобразуется в строку. Операции инкремента **--** или декремента **++** над строкой приведут её к числу. Унарный оператор плюс **+**



преобразует свой операнд в число, а унарный оператор отрицания **!** преобразует операнд в логическое значение.

example_9. Использование операторов

```
<script>
      console.log( true + "" );  // "true" = String(true)
      console.log( +"123" );  // 123 = Number("123")
      console.log( !"0" );  // false = Boolean("0")
</script>
```

Важно. Сколько раз не говори, что преобразование типов пригодится, сколько примеров не приводи – запомнить все правила достаточно сложно. Особенно, если JS не единственный язык программирования, который вы изучаете. Выход простой, использование **справочников** открытых источников.

Сейчас главное понять – есть такое событие, **приведение типов**, и оно может вносить поправки в вычисления выражений JS.

Самостоятельные работы:

- Задачи по приведению типов.
- Создание и инициализация массива с приведением значений к определенному типу.
- Создание и инициализация объекта с приведением свойств к определенному типу.
- Интерполяция значений свойств объектов в строки.

